StarUI Platform — Architecture

Monorepo layout, layer model, and package catalog

StarUI Platform Team

2026-05-15

Contents

1 Executive summary

StarUI is a consolidated monorepo that formalises a set of earlier proofs-of-concept and brings them under a single npm 10 workspace in the `@starui/*` namespace. It is a config-driven UI framework for capital-markets trading apps running on OpenFin.

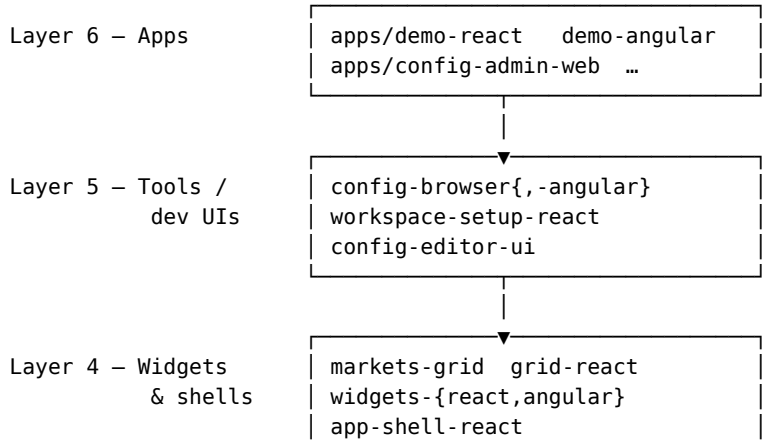
The platform's central design choice is the **runtime/framework matrix**: every capability is layered so a vanilla TypeScript core can be reused unchanged across React and Angular hosts, and across OpenFin and pure-browser runtimes. Every package in this document fits into one cell of that matrix, with strict import-boundary rules that prevent the layers from contaminating each other.

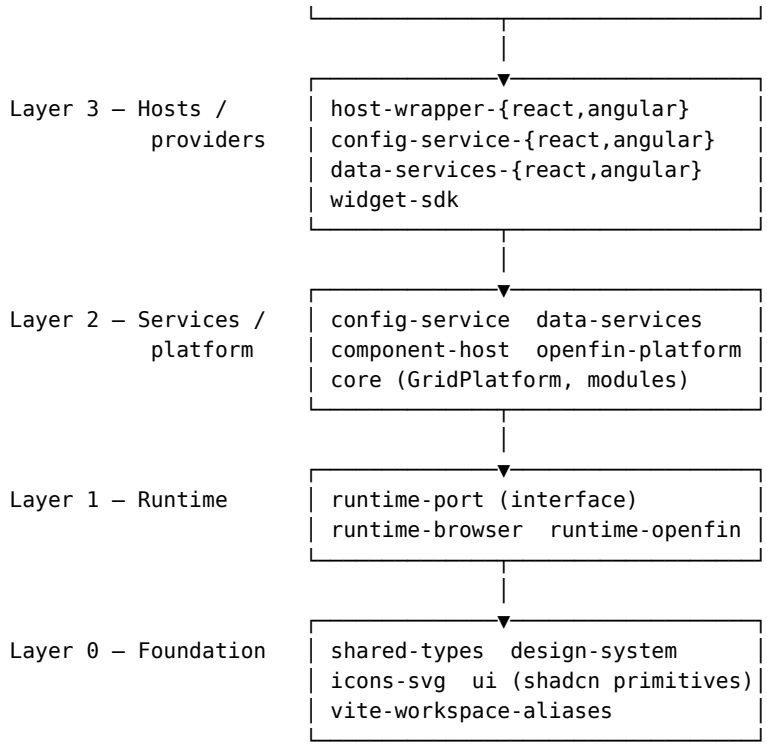
The repo currently produces 29 publishable packages (the tarballs in `libs/`) plus six runnable apps under `apps/`.

> **React is the lead surface.** The Angular packages (`widgets-angular`, `config-browser-angular`, `host-wrapper-angular`, `config-service-angular`, `data-services-angular`) **do not yet have feature parity with their React siblings** — Angular parity is an active **work in progress**. New capabilities land in React first, and Angular catches up package-by-package once the React surface stabilises. Treat the Angular bucket as preview-quality unless a specific package's README says otherwise.

2 Layer model

The repo is organized as a stack of layers. A package may import *only* from its own layer or layers below it.





Rules

- Foundation packages (`shared-types`, `design-system`, `tokens-primeng`, `icons-svg`) may not import from anywhere except each other.
- `core` may not import from any framework adapter (`widgets-angular`, `widgets-react`, `grid-react`).
- React and Angular siblings (`widgets-react` ↔ `widgets-angular`) may not import each other; they share the vanilla core.
- Only `runtime-openfin` and `openfin-platform` may import from `@openfin/core`.
- Apps import from packages, never the reverse.

3 Workspace organization

The `packages/` tree is split by **framework bucket**, then by **role** within that bucket:

```
packages/
├── shared/                  # vanilla TS, framework-agnostic
│   ├── core/                # grid platform, profile mgr, expression engine
│   ├── foundation/          # pure leaves (types, design tokens, icons)
│   ├── runtime/             # port + browser/openfin implementations
│   ├── services/            # config-service, data-services, component-host
│   └── platform/            # openfin-platform shell
│
├── react/                   # React-only packages
│   ├── ui/                  # shadcn primitives (singleton — no -react suffix)
│   ├── sdk/widget-sdk/      # widget contract (singleton)
│   ├── widgets/             # markets-grid, grid-react, widgets-react
│   ├── hosts/               # host-wrapper-react, app-shell-react
│   ├── providers/           # data-services-react, config-service-react
```

```
│   └── tools/              # config-browser-react, workspace-setup-react,
│                           #   config-editor-ui
│
└── angular/                # Angular-only packages (parity catching up)
    ├── widgets/            # widgets-angular
    ├── hosts/              # host-wrapper-angular
    ├── providers/          # data-services-angular, config-service-angular
    └── tools/              # config-browser-angular
```

Three rules govern this layout:

1. Pick the **framework bucket** by peer dep: vanilla TS → `shared/`, React → `react/`, Angular → `angular/`.
2. Pick the **sub-bucket** by role: foundation leaf, runtime, service, host, provider, sdk, widget, tool, or platform shell.
3. The package name carries the framework suffix **only when a twin can exist** (`config-browser-react` / `config-browser-angular`); singletons drop the suffix (`ui`, `widget-sdk`, `tokens-primeng`).

4 Build, test, and tooling

- **Package manager:** npm 10 workspaces. Never `pnpm` or `yarn`. Plain `npm ci` resolves cleanly — no `--legacy-peer-deps`.
- **Build orchestrator:** Turborepo 2 (`turbo build`, `turbo typecheck`, `turbo test`, `turbo e2e`).
- **Compile:** every library uses `rimraf dist && tsc` (or `ng-packagr` for Angular). The `rimraf` prefix defeats a TS5055 collision with Turbo's cache-restore.
- **Unit tests:** Vitest 4 + jsdom 29 (baseline 653 passing).
- **E2E tests:** Playwright 1.59 against `apps/demo-react` (baseline 195/214 passing — pre-existing failures tracked in `docs/E2E_STATUS.md`).
- **Version pinning:** stable lines per major (React 19.2.x, Angular 21.1.x, `@openfin/core` 43.101.x).

5 UI stack rules (non-negotiable)

Every UI component — new or updated — must:

1. **Consume `@starui/design-system` tokens.** No hardcoded colors, spacing, or typography; resolve through `--bn-*` / `--fi-*` CSS variables or the semantic exports.
2. **Use the framework-matching primitive library**: React → shadcn/ui via `@starui/ui`; Angular → PrimeNG themed via `@starui/tokens-primeng`. No native `<input>`/`<textarea>`/ `<select>` in React code.
3. **Be 100% dark/light compatible.** Every surface must render correctly under both `[data-theme="dark"]` and `[data-theme="light"]`.

6 Package catalog

The 29 packages currently produced as tarballs in `libs/`, organized by layer (bottom up). Each entry includes the layer, framework peer, and a concise summary of responsibilities.

6.1 Layer 0 — Foundation

6.1.1 `@starui/shared-types` (1.0.0)

Vanilla TypeScript type and constant definitions shared by every other package in the platform. Foundation leaf — imports nothing, has no peer dependencies. Owns the canonical shapes for configuration, data-provider contracts, widget identity, and profile state, so a change to a domain shape ripples through one file rather than four codebases.

6.1.2 `@starui/design-system` (0.1.0)

The single source of truth for visual design across React and Angular. Exports semantic CSS variables (`--bn-*`, `--fi-*`), a Tailwind preset, PrimeNG theme, shadcn adapter, and AG-Grid theme adapter via subpaths (`./css`, `./tailwind`, `./primeng`, `./shadcn`, `./adapters/ag-grid`). Consolidates the fi-trading-terminal token system and stern-2 UI wrappers. Every themed surface in the repo flows through this package.

6.1.3 `@starui/icons-svg` (1.0.0)

Framework-agnostic SVG icons for capital-markets apps, with prebuilt React and Angular bindings under subpath exports (`./react`, `./angular`, `./all-icons`, `./svg/*`). Peers include both `react` and `@angular/core` so either host gets zero-config tree-shakable icon access.

6.1.4 `@starui/ui` (1.0.0)

The StarUI shadcn/Radix primitive library, themed via `@starui/design-system`. Wraps the full Radix surface (accordion, dialog, dropdown, popover, tabs, tooltip, etc.) into shadcn-style components. Per platform convention, React UI code uses this package as its first stop instead of native `<input>`/`<select>`/`<textarea>` elements.

6.1.5 `@starui/vite-workspace-aliases` (1.0.0)

Build-time helper that auto-discovers `@starui/*` workspace packages and emits Vite alias entries mapping every declared export to its `src/` file. Eliminates per-app `vite.config.ts` alias drift — every app that calls the helper picks up new packages automatically.

6.2 Layer 1 — Runtime

6.2.1 `@starui/runtime-port` (0.1.0)

Pure interface package defining the runtime abstraction. Foundation-tier (no runtime imports, no peer deps). Every host that needs to read identity, theme, or lifecycle events depends on this interface so the concrete runtime can be swapped between OpenFin and browser without touching consumer code.

6.2.2 `@starui/runtime-browser` (0.1.0)

Browser implementation of `runtime-port`. Identity flows through URL parameters; theme is sourced from `prefers-color-scheme` + `[data-theme]`; popouts use `window.open`. The default runtime for demo apps and storybook.

6.2.3 `@starui/runtime-openfin` (0.1.0)

OpenFin implementation of `runtime-port`. Reads identity from the view's `customData`, bridges `fin.*` lifecycle events into the port surface, and falls back to `runtime-browser` for any capability the OpenFin shell does not provide.

6.3 Layer 2 — Services & platform

6.3.1 `@starui/core` (0.1.0)

The framework-agnostic vanilla-TS heart of the platform — `GridPlatform`, `ProfileManager`, `HistoryStack`, `ExpressionEngine`, module registry, persistence pipeline. Peers on `ag-grid-community`/`-enterprise` so the same business logic drives React and Angular surfaces. The import-boundary rules forbid it from importing any React or Angular adapter.

6.3.2 `@starui/config-service` (1.0.0)

Dual-mode configuration service for StarUI: Dexie/IndexedDB in local-dev mode, REST + Dexie in production mode. Owns the ConfigManager + ApplicationContext primitives and the schemas for roles, permissions, user profiles, and the application registry. Single peer is `@starui/core`.

6.3.3 `@starui/data-services` (0.1.0)

SharedWorker-backed data services runtime — one network connection per provider, many consumers multiplexed over it. Exposes `./runtime`, `./runtime/sharedWorker`, and `./runtime/client` subpaths so apps can mount the worker or attach a client independently. Peers on `@stomp/stompjs` for STOMP-based market-data providers.

6.3.4 `@starui/component-host` (1.0.0)

Lightweight component-hosting wrapper that injects OpenFin platform services (identity, config, theme) into registered components. Exports `./react` and `./angular` adapter subpaths, making it the bridge between the framework-agnostic platform and the per-framework hosts.

6.3.5 `@starui/openfin-platform` (1.0.0)

The OpenFin Workspace platform shell — bundles `@openfin/core`, `@openfin/workspace`, and `@openfin/workspace-platform` into a single configurable entrypoint and provides a `./config` subpath for platform-config helpers. Only this package and `runtime-openfin` are permitted to import from `@openfin/core`.

6.4 Layer 3 — Framework hosts & providers

6.4.1 `@starui/widget-sdk` (1.0.0)

The Stern Widget SDK — `useWidget` hook, `WidgetHost`, and the `PlatformAdapter` extensibility interface. Defines the contract that every hostable React widget implements. Sibling-free singleton (no Angular twin yet) because the Angular host stack embeds the same contract directly via `host-wrapper-angular`.

6.4.2 `@starui/host-wrapper-react` (0.1.0)

React `HostWrapper` component + `HostContext` + `useHost` hook. The single component-side seam (Seam #2 in `docs/ARCHITECTURE.md`) through which hosted components read identity, configManager, theme, and runtime lifecycle. Exports a `./test-bridge` subpath for mocking the host context in unit tests.

6.4.3 `@starui/host-wrapper-angular` (0.1.0)

Angular twin of `host-wrapper-react` — `HostService` injectable mirroring the same surface (identity, configManager, theme, lifecycle). Built with `ng-packagr`; ships as fesm2022. Lets Angular components participate in the same hosting contract as React siblings.

6.4.4 `@starui/config-service-react` (0.1.0)

React Provider + hook for `@starui/config-service`. Wires ConfigManager, ApplicationContext, and MarketsGrid storage into the React tree in ≤8 host lines. Peers on `react` and `@starui/data-services-react` so the config + data wiring lights up together.

6.4.5 `@starui/config-service-angular` (0.1.0)

Angular adapter for `@starui/config-service` — `provideConfigService()` factory and `ConfigServiceClient` injectable. Mirrors the React provider over the same vanilla core, so a config shape change touches `config-service` once and both hosts pick it up.

6.4.6 `@starui/data-services-react` (0.1.0)

React hooks for the SharedWorker data runtime — `DataServicesProvider`, `useProviderStream`, `useProviderStats`, `useResolvedCfg`, `useAppDataStore`, and `useDataProvidersList`. Exposes a `./runtime` subpath for direct runtime imports when a component needs to bypass React context.

6.4.7 `@starui/data-services-angular` (0.1.0)

Angular adapter for the data services runtime — `provideDataServices()` factory + `DataServicesService` + `injectAppData` / `injectProviderStream` / `injectDataProviderConfig` / `injectResolvedCfg`. Sibling parity with the React adapter; both depend on the same vanilla `data-services` package.

6.5 Layer 4 — Widgets & app shells

6.5.1 `@starui/grid-react` (0.1.0)

The StarUI grid customizer — React UI primitives, hooks, and module panels that decorate `@starui/markets-grid`. Bundles the ExpressionEditor (Monaco-driven), ColumnSettingsPanel, CalculatedColumnsPanel, ConditionalStylingPanel, ColumnGroupsPanel, and the TemplatePicker / FormatterPicker surfaces. Peers on AG-Grid React, `@starui/ui`, and `@starui/design-system`.

6.5.2 `@starui/markets-grid` (0.1.0)

The React `MarketsGrid` widget — the platform's flagship surface. Composes `@starui/core` (platform), `@starui/grid-react` (customizer UI), and AG-Grid Enterprise into a single declarative component. Ships its own `./styles.css` subpath for app-level import. Trading apps mount this; the customizer UI rides along automatically.

6.5.3 `@starui/widgets-react` (1.0.0)

React widget library for StarUI — `HostedMarketsGrid`, trading primitives, the v2 provider editor and data-provider selector, plus shared toolbar/header building blocks. Subpath exports (`./v2/markets-grid-container`, `./v2/provider-editor`, `./v2/data-provider-selector`, `./hosted`) keep bundle splitting sane. The "production" composition layer apps depend on.

6.5.4 `@starui/widgets-angular` (1.0.0)

Angular sibling of `widgets-react` — currently the DockConfigurator and DataProviderEditor with parity catching up. Single `.` export, fesm2022 build. Depends on the vanilla `@starui/data-services` core through `data-services-angular`.

6.5.5 `@starui/app-shell-react` (0.1.0)

The React `AppShell` component — collapses the three-app boot order (apply theme → runtime → ConfigManager → DataServices → HostWrapper) into one declarative root. Apps opt in to providers they actually use; turning a provider off means it never appears in the tree.

6.6 Layer 5 — Tools & dev UIs

6.6.1 `@starui/config-browser` (1.0.0)

React admin UI for inspecting and editing the contents of `@starui/config-service` — roles, permissions, user profiles, the app registry, and MarketsGrid storage. Peers on `@starui/markets-grid` for shared rendering, and uses the shadcn-themed `@starui/ui` primitives.

6.6.2 `@starui/config-browser-angular` (1.0.0)

Angular twin of `config-browser` — same admin surface, rebuilt on PrimeNG primitives themed by `@starui/design-system`. Peers on `@angular/forms`, `ag-grid-angular`, and `primeng`. Both config-browsers read the same vanilla config-service so admin behavior stays consistent.

6.6.3 `@starui/config-editor-ui` (0.1.0)

Engine-agnostic React editors for the config-service auth tables (roles, permissions, user profiles, app registry). Used both by `config-browser` and by host apps that embed inline admin views. Peers on `@starui/widgets-react` for shared form/table primitives.

6.6.4 `@starui/workspace-setup-react` (1.0.0)

React workspace-setup tool — the configurator UI for assembling an OpenFin workspace from registered components, dock entries, inspector panes, and import flows. Bundled with the OpenFin platform shell so the setup experience is consistent across all apps that adopt StarUI.

7 Applications

The `apps/` tree consumes the package catalog above. Six apps ship today:

App	Stack	Purpose
`apps/demo-react`	React	Flagship demo + Playwright E2E target
`apps/demo-angular`	Angular	Angular-parity demo
`apps/demo-configservice-react`	React	Standalone config-service walkthrough
`apps/config-admin-web`	React	Production config admin (uses config-browser)
`apps/config-service-server`	Node	REST backend for the production config mode
`apps/markets-ui-react-reference`	React	Reference composition for downstream apps

8 Import-boundary summary

The rules below are enforced by convention today (ESLint enforcement is a follow-up PR):

```
foundation/*       → may import other foundation packages only
runtime-port       → no imports
runtime-browser    → runtime-port only
runtime-openfin    → runtime-port, runtime-browser, @openfin/core
core               → foundation/* (no framework adapters)
services/*         → foundation/*, runtime-port, core
openfin-platform   → services/*, runtime-openfin, @openfin/*
hosts/*            → services/*, runtime-port, framework primitives
providers/*        → services/*, hosts/*
widgets/*          → providers/*, hosts/*, core, ui (React only)
tools/*            → widgets/*, providers/*, hosts/*
apps/*             → anything above (never the reverse)
```

9 Conventions reference

- **File naming:** filename casing matches the file's primary export. PascalCase for classes/components, camelCase for utility modules, kebab-case only in Angular (Style Guide) and in shadcn-managed React subtrees.
- **Commit prefixes:** `feat(pkg):`, `fix(pkg):`, `chore:`, `docs:`, `test:`, `ci:`, `refactor(pkg):`.
- **Complexity ceilings:** 800 LOC per file, 80 LOC per function.
- **No versioned code:** never `v1/`, `v2/`, `legacy/` in paths; superseded code is deleted in the same change as its replacement.
- **Documentation:** every feature add/update/remove updates `docs/IMPLEMENTED_FEATURES.md` in the same commit (or an immediate `docs:` follow-up).